



University Institute of Engineering
Department of Computer Science & Engineering

Experiment-4

Student Name: Pranav Chuahan
Branch: BE CSE (cyber security)
Semester: 5th
Subject Name: ADBMS

UID: 24BCY70004
Section/Group: 24AIT_KRG-G2
Date of Performance: 27/07/2026
Subject Code: 24CSP-333

1. Aim of the practical: To practice SQL queries using **ORDER BY** and **CASE** statements to customize the sorting of records, and to apply **self-joins with aggregate functions** to generate manager-wise employee reporting summaries.

2. Questions

(A) Write You are given a table tbl_happiness that contains the ranking of happiness for different countries. Your task is to retrieve all the records from the table **while ensuring that "India" and "Sri Lanka" appear at the top, followed by the rest of the countries in their original ranking order.**

```
CREATE TABLE tbl_happiness
(
    sno INT PRIMARY KEY,
    rankings INT,
    country VARCHAR(50)
);

INSERT INTO tbl_happiness
VALUES
(1,1,'Finland'),
(2,2,'Denmark'),
(3,3,'Iceland'),
(4,4,'Israel'),
(5,5,'Netherlands'),
(6,6,'Sweden'),
(7,7,'Norway'),
(8,126,'India'),
(9,128,'Sri Lanka');

select * ,
CASE
    when country = 'India' then 1
    when country = 'Sri Lanka' then 2
    else 3
END as Derived_rank
from tbl_happiness
order by Derived_rank ,rankings ;
```

S.No	Rankings	Country
1	1	Finland
2	2	Denmark
3	3	Iceland
4	4	Israel
5	5	Netherlands
6	6	Sweden
7	7	Norway
8	126	India
9	128	Sri Lanka



University Institute of Engineering

Department of Computer Science & Engineering

Solution:

```
select * ,  
CASE  
  when country = 'India' then 1  
  when country = 'Sri Lanka' then 2  
  else 3  
END as Derived_rank  
from tbl_happiness  
order by Derived_rank ,rankings ;
```

sno [PK] inte	rankings integer	country character varyin	derived_ integer
8	126	India	1
9	128	Sri Lanka	2
1	1	Finland	3
2	2	Denmark	3
3	3	Iceland	3
4	4	Israel	3
5	5	Netherlands	3
6	6	Sweden	3
7	7	Norway	3

(B) Make all the rankings in descending order except **India** and **Sri Lanka**.

```
CREATE TABLE tbl_happiness  
(  
  sno INT PRIMARY KEY,  
  rankings INT,  
  country VARCHAR(50)  
);  
  
INSERT INTO tbl_happiness  
VALUES  
(1,1,'Finland'),  
(2,2,'Denmark'),  
(3,3,'Iceland'),  
(4,4,'Israel'),  
(5,5,'Netherlands'),  
(6,6,'Sweden'),  
(7,7,'Norway'),  
(8,126,'India'),  
(9,128,'Sri Lanka');  
  
select * ,  
CASE  
  when country = 'India' then 1  
  when country = 'Sri Lanka' then 2  
  else 3  
END as Derived_rank  
from tbl_happiness  
order by Derived_rank ,rankings ;
```



University Institute of Engineering

Department of Computer Science & Engineering

Solution:

```
select * ,  
CASE  
    when country = 'India' then 1  
    when country = 'Sri Lanka' then 2  
    else 3  
END as Derived_rank  
from tbl_happiness  
order by Derived_rank ,rankings DESC ;
```

sno [PK] inte	rankings integer	country character varyin	derived_ integer
8	126	India	1
9	128	Sri Lanka	2
7	7	Norway	3
6	6	Sweden	3
5	5	Netherlands	3
4	4	Israel	3
3	3	Iceland	3
2	2	Denmark	3
1	1	Finland	3

- (C) A company maintain employee information in a single table where each employee may report to another employee acting as their manager. Prepare a report that displays the details of every employee who supervises at least one other employee. The report should include the manager's employee ID, name, total number of direct reports, and the average age of those reporting employees. The average age should be rounded to the nearest whole number and the final result should be sorted by employee ID.

```
CREATE TABLE Employees  
(  
    employee_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    reports_to INT,  
    age INT  
);  
  
INSERT INTO Employees (employee_id, name, reports_to, age)  
VALUES  
(9, 'Hercy', NULL, 43),  
(6, 'Alice', 9, 41),  
(4, 'Bob', 9, 36),  
(2, 'Winston', NULL, 37);
```



University Institute of Engineering

Department of Computer Science & Engineering

Solution:

```
SELECT
    m.employee_id,
    m.name,
    COUNT(e.employee_id) AS reports_count,
    ROUND(AVG(e.age), 0) AS average_age
FROM Employees m
JOIN Employees e
    ON m.employee_id = e.reports_to
GROUP BY
    m.employee_id,
    m.name
ORDER BY
    m.employee_id;
```

employee_id [PK] integer	name character	reports_count bigint	average_age numeric
9	Hercy	2	39

3. Learning Outcomes: .

- Learned to use **ORDER BY** and **CASE** statements for custom sorting of query results.
- Understood how to perform **self-joins** to retrieve hierarchical employee-manager relationships.
- Applied **aggregate functions** (COUNT(), AVG(), ROUND()) along with **GROUP BY** for data summarization.
- Developed SQL queries to generate **manager-wise reports** and customized output based on given conditions.